Week 1 - Revision

IC foundations, design choices and pre-RTL methods

Four lectures plus Tutorial 1. The first logic-synthesis lecture belongs to Week 2, even though it appears in the existing Day 01 reading edition. This short guide follows the course week. [1]

1. ICs | 2. Styles | 3. Economics | 4. Partitioning | 5. HLS | 6. Trade-offs | 7. Numericals | 8. Unix/sources

1. IC structure and fabrication

An **integrated circuit (IC)** combines devices and interconnections on a semiconductor die. Silicon forms the substrate and device regions in conventional CMOS. Metals provide interconnect; dielectric separates conductors; a **via** joins selected metal layers. Two wires crossing in a top view need not connect if they occupy insulated layers with no via. More routing layers provide freedom, with resistance, capacitance and process rules still present. [2]

Term	Meaning and distinction
Ingot	Bulk crystal before thin wafers are sliced
Wafer	Thin substrate carrying many die sites
Die	One circuit region, later separated by dicing
Packaged chip	Die connected and protected for use in a system
Photomask	Pattern used during an exposure step
Photoresist	Temporary light-sensitive coating; not the final metal

Simplified pattern transfer: deposit the film, coat resist, expose through a mask, develop the resist, etch the exposed film and strip the resist. Lithography defines the resist pattern; etching transfers it. Different patterned layers normally need different masks and accurate alignment. [2]

Remember: Moore's observation concerns a historical growth trend in device integration. A doubling in transistor count does not by itself guarantee doubled clock frequency, halved energy or doubled application speed.

2. Implementation styles and figures of merit

Style	What is customized	Advantage	Limitation
Full custom	Device choices and detailed physical design	Greatest freedom for demanding blocks	High effort, verification cost and turnaround
Standard-cell ASIC	Network built from characterized cells	Automation with efficient application-specific hardware	Up-front design/mask cost; fabrication commitment
Traditional gate array	Mainly selected interconnect layers over a prepared base	Reduced customization effort in its intended flow	Less freedom than a fully customized implementation
FPGA	Configuration of prefabricated programmable resources	Fast implementation and field updates	Programmability overhead in unit cost, area, power or timing

These are implementation styles. **Fabless**, **foundry** and **integrated device manufacturer (IDM)** describe business responsibilities. A fabless company designs; a foundry fabricates; an IDM combines design and manufacturing roles. A **PDK** supplies process-related models, rules and data needed to design for a technology. A PDK is not the customer's functional specification. [2,3]

Power, performance and area (PPA) are competing measures. Specify performance as latency, throughput or frequency. Specify area as cell, core or die area. A smaller sum of cell areas does not alone prove a smaller routable die. Testability, reliability, cost and time to market can also decide the implementation. [3]

Advantage/disadvantage questions: identify the workload and production volume first. An FPGA may fit a changing low-volume product; a high-volume ASIC may recover a larger fixed investment through lower cost per unit. Neither choice is universally best.

3. Economics and numerical checkpoints

For a stated linear model, total cost is fixed development cost plus quantity times unit cost:

$$C = F + Nv, \qquad N_{BE} = \frac{F_A - F_B}{v_B - v_A}.$$

The graph intercept is fixed cost; its slope is unit cost. A positive crossover needs a meaningful positive quantity. Equal unit costs do not recover a higher fixed investment. These are teaching models, not current commercial quotations. [3]

Worked example: ASIC cost is 3,600,000 + 18N and FPGA cost is 600,000 + 78N. Fixed-cost difference is 3,000,000; saving per unit is 60. The crossover is **50,000 units**. Below it, FPGA cost is lower in this model; above it, ASIC cost is lower. Do not divide by either unit cost alone.

Yield connects manufacturing to unit cost. If a wafer costs 8,000, contains 400 gross dies and has 80% die yield, it gives 320 good dies. Wafer cost per good die is 25. If packaging and final test add 6 per good die, with no further losses, finished-unit cost is 31. The denominator is good dies, not all die sites.

Quantity	Calculation	Common error
Die yield	Good dies / gross dies	Inverting the fraction
Cost per good die	Wafer cost / good dies	Ignoring yield
Task energy	Average power times duration	Treating lower power as automatically lower energy
Frequency	Reciprocal of clock period	Mixing ns and ps

For the capacitive switching component, $P = \alpha C V^2 f$ when α counts average zero-to-one charging events per cycle. A voltage reduction to 0.8 of its original value gives a 0.64 power ratio if activity, capacitance and feasible frequency stay unchanged. Leakage and short-circuit power are separate effects.

4. Abstraction, partitioning and IP integration

Abstraction controls how much implementation detail a model exposes. An algorithm says what to compute; RTL makes register transfers and clocked behavior explicit; a gate netlist gives logical structure; layout adds physical geometry. Lowering abstraction usually adds detail and makes exploration more expensive. A high-level estimate is useful, but it is not final physical signoff. [4]

Hardware-software partitioning assigns functions to dedicated hardware or software on a processor. Hardware can exploit parallelism and tailored datapaths; software offers flexibility and easier updates. Consider acceleration benefit, transfer overhead, memory access, power, area, cost and engineering effort together.

Worked example: software takes 80 ms. An accelerator needs 8 ms computation plus 12 ms communication and coordination. The system now takes 20 ms, so speedup is 4, rather than the compute-only factor of 10. Accelerate the measured bottleneck and include interface costs.

A **system-on-chip (SoC)** integrates interacting processing, memory, communication and other functions. **Intellectual property (IP)** here means reusable design content, not a guarantee that every integration is correct. Soft IP is commonly supplied as synthesizable RTL; hard IP has a process-specific physical implementation. [5]

Reuse benefit	Integration obligation
Avoid rebuilding an established block	Check its configuration and documented operating assumptions
Reuse vendor verification effort	Verify the integrated system and its interfaces
Reuse an interface or subsystem	Match protocols, widths, clocks, resets and timing

Trap: verified blocks do not automatically make a verified SoC. A reset mismatch, unsafe clock crossing or incorrect handshake can break the integrated system while each block works in its original environment.

5. Behavioural synthesis: what it actually builds

Behavioural synthesis, also called **high-level synthesis (HLS)**, constructs a timed RTL architecture from an algorithmic description and implementation constraints. Typical supported inputs are algorithmic C/C++ or another tool-supported high-level language. The result contains a datapath, storage, control and interfaces. HLS generates hardware; it does not simply execute the program on a CPU. [5,7]

For `y = (a+b)+c`, the arithmetic requirement alone does not say whether to use one shared adder, two chained adders or two pipeline stages. The tool must make architectural decisions:

Decision	Meaning	Example
Allocation	Choose resource types and quantities	One adder or two
Scheduling	Assign operations to time steps	First addition in cycle 1, second in cycle 2
Binding	Associate operations and values with actual resources/storage	Both additions use adder U1 in different cycles
Control generation	Sequence operations and mark valid transfers	Select operands and assert output-valid at completion
Interface generation	Connect the architecture to its environment	Define input acceptance and output handshakes

Scheduling respects data dependencies, clock requirements, resource availability and directives. Binding and scheduling influence each other because different resources can have different latency and throughput. These are related architectural choices, not merely source-code renaming. [7]

Constraints matter: clock period, resource limits, latency, initiation interval, memory/interface bandwidth and power or area goals can alter the result. An unrestricted source-language feature is not automatically synthesizable. A finite circuit needs a supported implementation model.

Boundary: HLS produces RTL architecture. Logic synthesis subsequently maps RTL toward gates; physical design places and routes the implementation. HLS does not directly place transistor polygons or finish manufacture.

6. HLS advantages, limitations and architecture trade-offs

Advantages. A concise algorithmic model can reduce manual RTL effort. Changing constraints or directives can explore different architectures and resource/performance trade-offs. A high-level model also supports fast functional testing and reuse of suitable algorithms. These are productivity opportunities; results still depend on the source structure, tool and target. [8]

Limitations. Supported language subsets exclude some host-software operations. For example, AMD's documented hardware synthesis flow does not synthesize ordinary operating-system file operations or dynamic creation/freeing of hardware memory. A testbench can still use host-side operations. Memory ports, dependencies, numeric widths and interfaces constrain parallelism. Generated RTL still needs functional, protocol and implementation verification. Do not assume globally optimal PPA or automatic clock-domain safety. [7,9]

Architecture for (a+b)+c	Resources and schedule	Main trade-off
Chained	Two adders between input and output registers	One-cycle arithmetic, but two adder delays in the path
Shared	One adder reused across two steps	Saves an adder; needs muxing, stored operands and control
Pipelined	Two adders separated by a register	Shorter stage paths; increased cycle latency and extra registers

For the simple shared schedule, two additions require two adder uses. If that adder accepts one operation per cycle, the long-run initiation interval is at least two cycles per transaction. Pipelining cannot create an extra arithmetic operation per cycle from a resource that cannot supply it.

Keep transaction ownership. If the first step stores a+b, the later step must use c from the same accepted input tuple. Either capture c or require it to remain stable under a documented handshake. In a pipeline, delay c to match its partial sum. Reset and valid signals must establish which result belongs to which request.

What to compare: use latency in time, steady-state throughput, resource/area use, power and the interface contract. Fewer cycles do not necessarily mean less time if the required clock period is longer.

7. Latency, throughput and timing worked examples

Latency is elapsed time from a defined accepted input to its corresponding valid output. **Initiation interval (II)** is spacing between accepted independent transactions. At period T, cycle latency L gives time latency LT; steady-state throughput is ideally $1/(II\,T)$ for one item per initiation. Stalls and bandwidth can lower it. [5,7]

Example A: compare the right quantities. Architecture A takes 2 cycles at 12 ns, with II=1. Its latency is 24 ns and ideal throughput is about 83.3 million items/s. Architecture B takes 1 cycle at 20 ns, with II=1. Its latency is 20 ns and throughput is 50 million items/s. B has lower single-item latency; A has greater steady-state throughput.

Example B: pipeline completion time. A pipeline has latency 3 cycles and II=1. Five independent items accepted on consecutive opportunities finish after $3 + (5 - 1) \times 1 = 7$ cycles from the first acceptance under this cycle-count convention. State the acceptance/capture convention when interpreting a waveform.

For a simple register-to-register setup budget with zero skew and uncertainty:

$$T \geq t_{CQ,\,max} + d_{comb,\,max} + t_{setup}.$$

With clock-to-Q 0.8 ns, combinational delay 5.1 ns and setup time 0.6 ns, the required period is at least 6.5 ns, corresponding to about 153.8 MHz in this model. The 5.1 ns combinational delay alone is not the full clock budget. This introduces the timing meaning used in the Week 1 HLS lecture; detailed STA comes later.

Self-check, without looking back: explain why sharing needs control; distinguish scheduling from binding; explain how II differs from latency; state why software may be the better choice for changing functionality. These prompts do not reveal the assignment choices.

8. Unix quick revision and source trail

Need	Command or concept	Detail to retain
Locate work	`pwd`, `cd`, `ls -a`	Relative paths depend on the current directory
Create/copy/move	`mkdir`, `touch`, `cp`, `mv`	touch normally preserves existing contents
Read/help	`cat`, `man`, `which`, `type`	A shell built-in differs from an external executable
Storage	`du -h`, `df -h`	Directory usage differs from filesystem free space
Processes/jobs	`ps`, `top`, `jobs`	PID and shell job number are different identifiers
Job control	Ctrl+Z, `bg`, `fg`, trailing `&`	Suspended means stopped; background can still execute

Quote paths with spaces. Linux names are generally case-sensitive. Background execution alone does not guarantee survival after terminal closure. `sudo` changes privileges; it does not fix a wrong path or syntax for a different interpreter. [6]

Sources and further reading

1. Official NPTEL course and weekly outline.

2. Lecture: IC concepts I.

3. Lecture: IC concepts II.

4. Lecture: design flow I.

5. Lecture: design flow II.

6. Unix tutorial; GNU Bash job control.

7. AMD UG1399: scheduling and binding.

8. AMD UG1399: HLS benefits.

9. AMD UG1399: C/C++ synthesis support.

The 600 practice MCQs are newly authored study questions, not verified past examination questions. A student exam discussion reports assignment-style theory and numerical questions; this is anecdotal and supplies no authenticated Week 1 exam paper. It motivates a mixed practice format, not a prediction.

Full lecture figures and original handwritten notes remain in the Day 01 reading PDF. Original assignments are taken from Kapil's enrolled 2026 NPTEL course; the practice bank is separate.